CSI2120: PROGRAMMING PARADIGMS
CSI2520: PARADIGMES DE PROGRAMMATION

Lab Go 1:

 # 1:

Écrire une fonction Go qui prend comme paramètre une variable de type `float32` et retourne deux variables de type entier. Le premier entier est l'arrondissement inférieur (*floor value*) du nombre réel et le deuxième entier est l'arrondissement supérieur (*ceiling value*) de ce même nombre réel. Démontrer le bon fonctionnement de cette fonction à partir d'une fonction `main`.

Write a Go function that takes a parameter of type `float32` and returns two integer values. The first integer must be the floor value of the real number, and the second integer must be the ceiling value of that same real number. Demonstrate that the function works correctly by calling it from a `main` function.

 # 2:

Écrire une fonction qui retire tous les nombres négatifs dans un slice d'entiers. La fonction retourne un nouveau slice d'entiers avec seulement les nombres positifs. Assurez-vous que le slice retourné a une capacité égale au tableau reçu. Démontrer le bon fonctionnement de cette fonction à partir d'une fonction `main`.

Write a function that removes all negative numbers from a slice of integers. The function must return a new slice containing only the positive numbers. Ensure that the returned slice has the same capacity as the original slice. Demonstrate that the function works correctly by calling it from a `main` function.

#3:

Voici ci-dessous un arbre binaire contenant des instances de type `Point` insérées dans l'arbre dans un ordre quelconque (ce n'est pas un arbre binaire de recherche).

I. Écrire la méthode permettant d'imprimer le contenu d'un tel arbre de façon postordre.
II. Écrire une méthode `find(x,y)` permettant de déterminer si un point donné se trouve dans un de ces arbres
III. Créer l'interface `PointSearcher` spécifiant la méthode `find`
IV. Tester vos méthodes avec la fonction `main` de la page suivante et devant produire la sortie suivante :

```
(2,2)(1,6)(8,3)(5,1)(6,4)(0,9)(7,2)(3,6)(4,7)(2,3)
Found: 7 2
Not Found
```

Below is a binary tree containing instances of type `Point`, inserted in an arbitrary order (this is not a binary search tree).

I. Write a method that prints the contents of this tree using a post-order traversal.
II. Write a method `find(x, y)` that determines whether a given point is present anywhere in the tree.
III. Create an interface `PointSearcher` that specifies the `find` method.
IV. Test your methods using the `main` function on the next page, and ensure that your program produces the expected output shown.

```
(2,2)(1,6)(8,3)(5,1)(6,4)(0,9)(7,2)(3,6)(4,7)(2,3)
Found: 7 2
Not Found
```

```go
type Point struct {
     x int
     y int
}

type PtTree struct {
     pt Point
     left, right *PtTree
}
```

```go
func main() {
     tree := PtTree{Point{2, 3},
               &PtTree{Point{5,1},
               &PtTree{Point{2,2},nil,nil},

     &PtTree{Point{8,3},&PtTree{Point{1,6},nil,nil},nil}},
               &PtTree{Point{4,7},
               &PtTree{Point{7,2},
               &PtTree{Point{6,4},nil,nil},
               &PtTree{Point{0,9},nil,nil}},
               &PtTree{Point{3,6},nil,nil}}}

     tree.postOrder()
     fmt.Println("")

     var ps *PointSearcher
     ps= &tree

     u, v := 7, 2
     if ps.find(u, v) {
          fmt.Printf("Found: %d %d \n", u, v)
     } else {
          fmt.Printf("Not Found\n")
     }

     x, y := 8, 6
     if ps.find(x, y) {
          fmt.Printf("Found: %d %d \n", u, v)
     } else {
          fmt.Printf("Not Found\n")
     }
}
```